

Time-series based bridge damage diagnosis using a novel graph transformer with vector quantization

Hung Tran Viet^a, Hoa Tran Ngoc^{a,*}, Bui Tien Thanh^a, Tuyen Vu Quang^b, Loc Bui Phuc^a

^a Faculty of Civil Engineering, University of Transport and Communications, Hanoi, Vietnam

^b Faculty of Information Technology, University of Transport and Communications, Hanoi, Vietnam

ARTICLE INFO

Keywords:

Time-series classification
Structural health monitoring
Graph transformer
Vector quantization
Ensemble learning

ABSTRACT

Structural Health Monitoring (SHM) is critical for infrastructure safety; however, interpreting the massive volume of time-series data generated by sensors remains a significant challenge. Conventional methods often struggle to efficiently process such data or fail to capture complex, long-range dependencies between structural components. To address this, we introduce a novel deep learning framework: the Graph Transformer with Vector Quantization and Ensemble Learning (GT-VQ-EL). Unlike standard approaches that treat sensors as isolated points, our method models the bridge as a graph, preserving the physical connectivity of sensors. The framework operates in three stages: (i) a Graph Transformer analyzes global vibration propagation, capturing spatiotemporal relationships across the structure; (ii) Vector Quantization discretizes continuous signals into stable codebook vectors, functioning as a regularization mechanism to mitigate environmental noise; and (iii) Ensemble Learning aggregates predictions from multiple sub-models via majority voting to ensure diagnostic reliability. We validated this approach using synthetic data generated from a calibrated finite element model of the Cua Rao Bridge in Vietnam. The GT-VQ-EL model achieved a test accuracy of 93.62%, significantly outperforming baseline methods such as standard Graph Neural Networks (69.03%), Graph Convolutional Networks (82.15%) and Quantized Graph Neural Networks (70.86%), while the Temporal Periodicity and Global Spatial Graph Neural Network reached 89.62%. These results demonstrate that combining graph-based contextual learning with robust noise filtering offers a highly accurate and scalable solution for monitoring complex civil infrastructure.

1. Introduction

In recent decades, SHM has become an important part of research on modern infrastructure [1,2]. This field has experienced rapid development, driven by advancements in sensor technologies, enhanced computational capabilities, and sophisticated data utilization strategies. Today, SHM plays a key role in keeping infrastructure safe and reliable, particularly within the transportation sector, where early detection of unusual behavior and maintenance plans based on real data are indispensable for ensuring structural safety, reducing service interruptions, and protecting public safety [3–6].

SHM systems primarily produce time-series data that reflect the dynamic behavior of structures under different operational and environmental conditions [7,8]. These data streams, including measurements such as acceleration, displacement, strain, and stress, are collected by sensor networks placed at important parts of the structure,

allowing continuous and detailed monitoring over time. Although such information is very useful for assessing the condition of the structure, it is also difficult to analyze because there is a large amount of data, many variables, and the signals are often noisy. Conventional processing techniques often cannot fully extract important information from these complex datasets. This has fueled increasing interest in advanced data-driven approaches that can handle large time-series datasets, discover hidden patterns, and more reliably assess structural behavior and damage over time [9].

The rapid advancement of machine learning (ML) models in recent years has brought many new developments in SHM applications [10, 11]. ML-based approaches offer numerous advantages, particularly in their ability to work with large and complex datasets, reduce manual effort in data analysis, and improve the efficiency, noise resistance, and reliability of structural assessment systems. Among various ML techniques, deep learning (DL) has emerged as a powerful approach because

* Corresponding author.

E-mail address: ngochoa@utc.edu.vn (H.T. Ngoc).

<https://doi.org/10.1016/j.knosys.2025.115248>

Received 10 September 2025; Received in revised form 5 December 2025; Accepted 28 December 2025

Available online 30 December 2025

0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

it can automatically learn useful features directly from raw data [12]. DL models have demonstrated outstanding performance in modeling complex, multivariate time-series signals, with successful implementations across multiple domains including industrial process monitoring, infrastructure inspection, and intelligent transportation systems [13, 14].

Graph Neural Networks (GNNs) are a class of DL models designed to work with graph-structured data, that is, data made of nodes and connections between them. Unlike traditional neural network architectures, they can effectively capture how different parts of a system are connected in space, making them well-suited for problems where the data do not lie on a regular grid. In SHM applications, GNNs represent structural systems as graphs, where nodes denote sensor locations and edges encode how structural components are physically connected or influence each other. This formulation allows the model to learn spatial relationships between different parts of the structure. Recent work by Yu Liu et al. [15] proposed the TPS-GNN model, which adds information about repeating time patterns and overall spatial behavior of the structure, achieving up to a 78 % improvement in predictive accuracy compared to conventional methods. Although TPS-GNN effectively captures temporal patterns, it relies on a standard message-passing mechanism. This approach often fails to model long-range dependencies, particularly in sensor networks with irregular layouts. Furthermore, processing continuous-valued features makes the model susceptible to the high-frequency noise.

To improve the capacity of GNNs in capturing localized structural behavior, Graph Convolutional Networks (GCNs) have been developed as a specialized variant that performs convolutional operations on graph-structured data. By leveraging spectral graph theory, GCNs aggregate and transform features from neighboring nodes, enabling more stable and efficient learning of spatial relationships. Within SHM, GCNs have demonstrated strong performance in modeling the spatial influence of distributed sensor nodes, making them well-suited for tasks such as damage localization and anomaly detection [16]. Compared to standard GNNs, they offer enhanced feature integration and generalization, particularly in scenarios with limited training data. However, like other spatial models, conventional GCNs do not explicitly account for temporal dependencies, which restricts their applicability in time-series-based SHM applications.

Quantized GNNs represent a computationally efficient extension of conventional architectures, in which node features and model parameters are encoded using low-precision formats to reduce memory usage and accelerate inference. This quantization process also acts as an implicit regularizer, helping to mitigate overfitting, particularly in cases involving limited or noisy data. Despite these advantages, standard quantized GNNs often rely on static codebooks, which limit their adaptability and may cause significant information loss. As noted by Kose et al. [17], such static schemes can severely compromise information fidelity under dynamic or heterogeneous input conditions, thereby constraining the model's expressive power in practical SHM applications.

Recently, hybrid architecture combining Transformers and Vector Quantization have emerged in Computer Vision and NLP domains (e.g., Transformer-VQ [18]). However, directly applying these models to SHM presents significant challenges. First, standard Transformers operate on Euclidean data, failing to adequately capture the non-Euclidean graph topology characteristic of sensor networks on complex infrastructure. Moreover, conventional VQ methods typically employ static codebooks that neglect the continuous temporal dynamics inherent in structural vibrations.

To address the limitations of current SHM diagnostic frameworks in modeling large-scale spatiotemporal responses, we propose a new DL architecture called GT-VQ-EL. The framework operates as a three-stage pipeline. First, the structure is modeled as a graph, in which sensors or structural components are treated as nodes and their physical or geometric relationships are treated as edges. This graph is processed by a

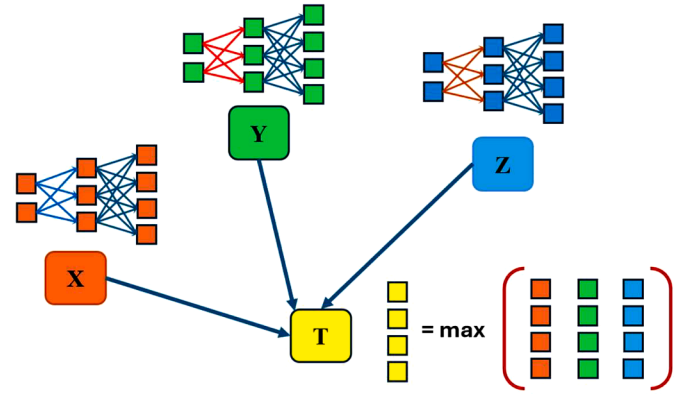


Fig. 1. Operational mechanisms of GNN.

Graph Transformer (GT) module that uses attention mechanisms and temporal ordering to capture long-range spatial and temporal interactions across the entire structure in an efficient way. Second, the high-dimensional features produced at each node are compressed by a vector quantization (VQ) module with an additional constraint that encourages smooth evolution over time. This step reduces model complexity, limits overfitting, and preserves consistent temporal behavior. Finally, the quantized features are passed to several parallel classifiers, and their individual predictions are combined using an ensemble learning (EL) strategy based on majority voting.

To clarify why these three components are combined, we consider a simple example of a bridge monitoring system affected by both structural damage (e.g., a crack) and environmental noise (e.g., wind). First, the GT is used because the bridge sensors are not arranged on a regular grid; they are connected according to the structural layout. By modeling the bridge as a graph, the GT can correctly capture how vibrations at one location propagate to connected members, rather than treating sensors as independent or only locally related. Second, VQ helps separate meaningful structural responses from noisy environmental effects. Instead of working with fully continuous and highly variable signals, VQ maps latent features into a finite set of typical states, which reduces random fluctuations caused by wind and other disturbances. Third, EL combines several classifiers that may have different strengths and weaknesses. By using majority voting across these classifiers, the framework reduces the risk that a single biased model will cause false alarms or missed detections, resulting in more stable and reliable damage diagnosis.

The proposed approach is validated using a dataset generated from a calibrated finite element model (FEM) of the Cua Rao Bridge, with a comprehensive performance comparison conducted against several DL baselines, including TPS-GNN, GNN, GCN, and their quantized variant. Experimental results reveal that GT-VQ-EL consistently outperforms these models in terms of classification accuracy, convergence speed, and robustness. These findings underscore the model's strong potential for deployment in real-world SHM applications.

This paper is organized as follows. Section 2 outlines the theoretical foundations of the DL models adopted in this study, including GNN, GCN, quantized GNN, and the proposed GT-VQ-EL. Section 3 details the dataset generation process, encompassing the simulation model and experimental model. Section 4 presents the experimental results, comparative evaluations, and robustness analyses. Finally, Section 5 summarizes the key findings, addresses the study's limitations, and suggests directions for future research.

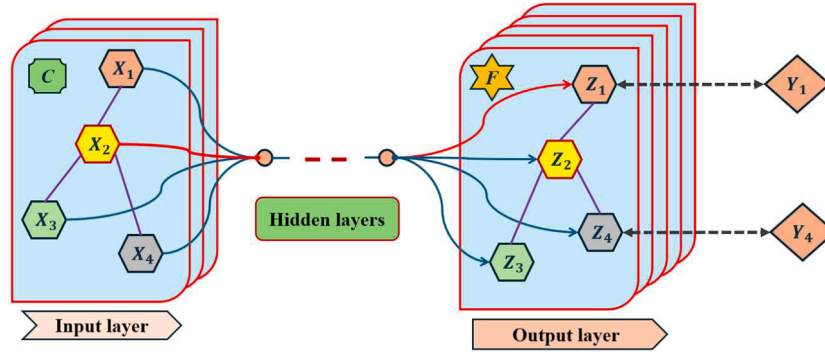


Fig. 2. GCN architecture.

2. Methods

2.1. Graph neural network (GNN)

First introduced by Scarselli et al. in 2009 [19], GNNs constitute a class of DL models specifically designed to process graph-structured data. Unlike traditional neural networks that assume independent and identically distributed samples, GNNs operate under a message-passing paradigm. In this framework, each node iteratively updates its representation by aggregating information from its local neighborhood, thereby capturing complex structural dependencies and feature interactions.

As illustrated in Fig. 1, the model operates by transmitting latent information across the nodes and edges of the graph. A key advantage of this approach lies in its ability to simultaneously integrate node-level attributes and the global graph topology. By stacking multiple layers, the network extends its receptive field, allowing the final embeddings to retain information about both local characteristics and long-range structural patterns [20].

2.2. Graph convolutional network (GCN)

Introduced by Thomas Kipf and Max Welling [21] in 2017, GCNs apply a first-order approximation of localized spectral filters to graph-structured data. Conceptually, they extend traditional CNNs to non-Euclidean domains by enabling convolutional operations on graphs [20]. As a specialized variant within the broader GNN framework, GCNs are optimized for processing graph-based inputs and have demonstrated strong performance in tasks such as node classification, graph classification, and edge prediction.

Consider a batch of A graph samples, each consisting of M nodes. Every node is associated with a feature vector, and the complete set of node features forms a matrix $X \in \mathbb{R}^{M \times G}$, where G denotes the number of feature dimensions. The structural connectivity among nodes is encoded by an adjacency matrix $B \in \mathbb{R}^{M \times M}$, which represents pairwise relationships. Both X and B serve as inputs to the model.

A GCN layer is specifically designed to operate on such graph-structured data. The layer propagates information using a spectral-based rule, defined as follows:

$$H^{(l+1)} = \sigma \left(\tilde{G}^{-\frac{1}{2}} \tilde{B} \tilde{G}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (1)$$

where:

$\tilde{B} = B + I$ is the adjacency matrix with added self-loops, and I is the identity matrix.

$\tilde{G}$ is the degree matrix of $\tilde{B}$, with diagonal entries defined as:

$$\tilde{G}_{ii} = \sum_j B_{ij} \quad (2)$$

$H^{(l)}$ denotes the node feature matrix at the l^{th} layer, with $H^0 = X$ as the input feature matrix.

$W^{(l)}$ is the trainable weight matrix at layer l .

$N \sigma(\cdot)$ is a nonlinear activation function, such as ReLU or softmax, applied element-wise.

As illustrated in Fig. 2, the final feature representation of each node is denoted by Z , obtained after passing the input features X through one or more hidden layers. The node connectivity is encoded by the adjacency matrix B (or its normalized form $\tilde{B}$) is consistently applied across all layers, regardless of network depth). This shared structure ensures that the spatial relationships among nodes are preserved throughout the learning process.

Assuming a two-layer GCN architecture is used, with ReLU as the activation function in the first layer and softmax in the second, the overall forward propagation is defined as:

$$Z = f(X, B) = \text{softmax}(\hat{B} \text{ReLU}(\hat{B} X W^{(0)}) W^{(1)}) \quad (3)$$

where:

- $X \in \mathbb{R}^{M \times G}$: input feature matrix
- $\hat{B} = \tilde{G}^{-\frac{1}{2}} \tilde{B} \tilde{G}^{-\frac{1}{2}}$: symmetrically normalized adjacency matrix,
- $W^{(0)}, W^{(1)}$: trainable weight matrices for the first and second layers, respectively,
- ReLU: element-wise rectified linear unit activation function,
- softmax: applied row-wise to produce a probability distribution over classes for each node,
- $Z \in \mathbb{R}^{M \times C}$: output matrix of predicted class probabilities, where C is the number of classes.

In the final step, a cross-entropy loss function is applied to all labeled nodes to optimize the model. The loss is computed as:

$$L = - \sum_{l \in y_L} \sum_{f=1}^F Y_{lf} \ln Z_{lf} \quad (4)$$

where:

- y_L is the set of indices corresponding to labeled nodes,
- F is the number of output classes,
- $Y_{lf} \in \{0, 1\}$ is the ground truth label indicator (1 if node l belongs to class f , and 0 otherwise),
- $Z_{lf} \in [0, 1]$ is the predicted probability for node l belonging to class f , as output by the softmax layer.

This loss function penalizes the divergence between predicted and true class distributions, and the summation is only performed over nodes

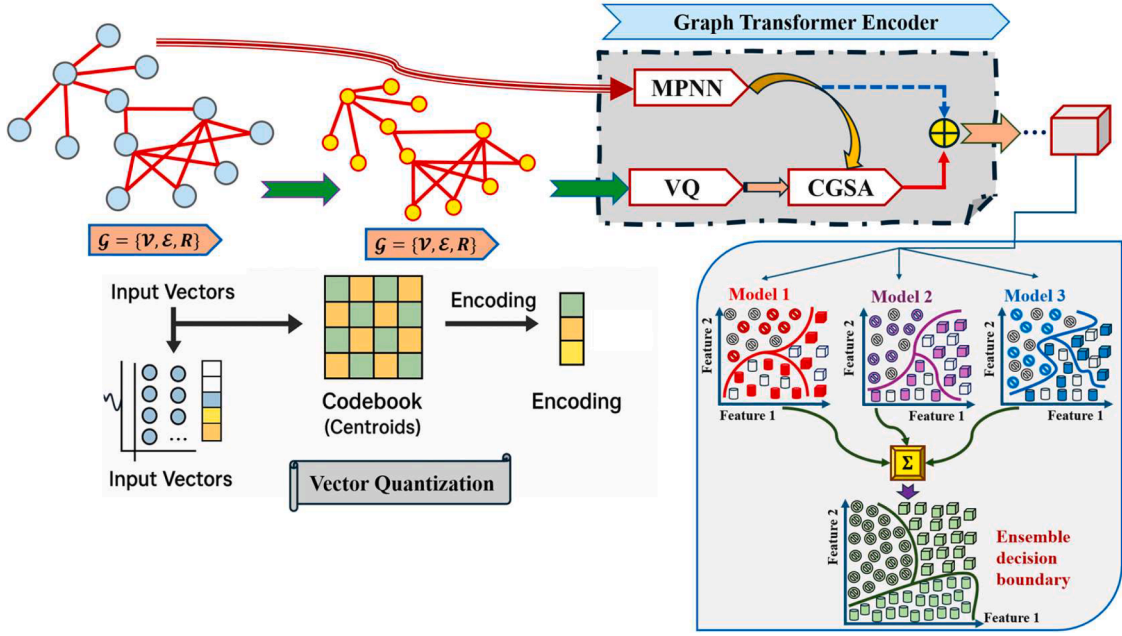


Fig. 3. The overview of GT-VQ-EL model.

with available labels.

2.3. Quantization GNN [22]

A quantized GNN architecture utilizes distributed graph filters to process signals supported on graphs. At layer $\ell = 1$, the input to the GNN is a graph signal $x_0^1 = x$. At each subsequent layer $\ell > 1$, the GNN receives $F_{\ell-1}$ input features $\{x_{\ell-1}^g\}_{g=1}^{F_{\ell-1}}$ from layer $\ell - 1$ and produces F_{ℓ} output features $\{x_{\ell}^g\}_{g=1}^{F_{\ell}}$. Each input feature $x_{\ell-1}^g$ for $g = 1 \dots F_{\ell-1}$ is quantized and independently processed by F_{ℓ} graph filters in parallel.

For a GNN utilizing a node-invariant graph filter, consider the k^{th} shifted version of the input feature at layer $\ell - 1$, denoted by $x_{\ell-1}^{g(k)} = S^k x_{\ell-1}^g$, which represents the input feature $x_{\ell-1}^g$ after being shifted k times via the graph shift operator S , i.e., exchanged with k -hop neighbors at layer $\ell - 1$. The quantized representation of this shifted feature is given by $\tilde{x}_{\ell-1}^{g(k)} = Q(x_{\ell-1}^{g(k)}) = x_{\ell-1}^{g(k)} + e_{\ell-1}^{g(k)}$, where $Q(\cdot)$ denotes the quantization function and $e_{\ell-1}^{g(k)}$ represents the quantization error.

The quantized signal is then exchanged with neighboring nodes, resulting in the quantized shifted signal $x_{\ell-1}^{g(1)} = S \tilde{x}_{\ell-1}^{g(0)} = S(x_{\ell-1}^{g(0)} + e_{\ell-1}^{g(0)})$. This intermediate signal $x_{\ell-1}^{g(1)}$ is subsequently quantized again, yielding $\tilde{x}_{\ell-1}^{g(1)}$ which is then transmitted to neighboring nodes. This iterative process of shifting, quantization, and communication is repeated for K steps.

The outputs of the node-variant graph filters are aggregated over the input feature index g to produce the f^{th} convolved feature:

$$\tilde{z}_{\ell}^f = \sum_{g=1}^{F_{\ell-1}} \left(\sum_{k=0}^K \text{diag}(h_{kl}^g) S^k x_{\ell-1}^g + \sum_{k=1}^K \text{diag}(h_{kl}^g) \sum_{\kappa=0}^{k-1} S^{k-\kappa} e_{\ell-1}^{g(\kappa)} \right) \quad (5)$$

for $f = 1 \dots F_{\ell}$

The outputs of the edge-variant graph filters are aggregated over the input feature index g to compute the f^{th} convolved feature:

$$\tilde{z}_{\ell}^f = \sum_{g=1}^{F_{\ell-1}} \left(\sum_{k=1}^K \left(\prod_{\kappa=1}^k \Psi_{\kappa}^g \right) x_{\ell-1}^g + \sum_{k=1}^K \sum_{\kappa=0}^{k-1} \left(\prod_{\tau=\kappa+1}^k \Psi_{\tau}^g \right) e_{\ell-1}^{g(\kappa)} \right) \quad (6)$$

for $f = 1 \dots F_{\ell}$

The quantized convolved feature $\tilde{z}_{\ell}^f$ is passed through a nonlinear activation function $\sigma(\cdot)$ to produce the f^{th} output of the convolutional layer:

$$x_{\ell}^f = \sigma(\tilde{z}_{\ell}^f) = \sigma \left(\sum_{g=1}^{F_{\ell-1}} \left(H_{\ell}^g(S) x_{\ell-1}^g + e_{\ell-1}^g \right) \right) \quad (7)$$

for $f = 1 \dots F_{\ell}$

The output feature of the final convolutional layer is denoted by x_L . This output depends on the input graph signal X and the sequence of quantized filter banks applied throughout the network. By introducing the quantized filter tensor $\tilde{\mathcal{H}}(S)$, which encapsulates all the quantized filters across layers, the entire GNN can be interpreted as a mapping $\tilde{\Phi}(\cdot)$ that takes the input signal x and the filter tensor $\tilde{\mathcal{H}}(S)$ as inputs, and produces the output:

$$\tilde{\Phi}(X; \tilde{\mathcal{H}}(S)) = x_L \quad (8)$$

2.4. Graph transformer with vector quantization - ensemble learning (GT-VQ-EL)

This study introduces GT-VQ-EL, a unified DL framework specifically engineered for time-series-based SHM. As illustrated in Fig. 3, the architecture consists of three integrated modules: aGT encoder for spatiotemporal feature extraction, aVQ module for noise filtering via discrete representation, and anEL mechanism for robust classification.

First, the bridge is conceptualized as a graph in which sensors or structural components serve as nodes and their physical or functional interrelations are modeled as edges. Within this representation, GT encoder captures long-range spatio-temporal dependencies using distance-aware attention bias, causal masking, and positional encoding, enabling efficient global context learning across the entire structure. VQ module then discretizes the high-dimensional node embeddings through an adaptive codebook with temporal smoothness regularization, effectively reducing overfitting, compressing latent representations, and ensuring consistent temporal dynamics. Finally, an EL layer aggregates the outputs of multiple diverse classifiers, each trained on the quantized embeddings, to form a robust decision boundary that enhances prediction accuracy and generalization.

VQ was originally introduced by Shannon [23] in 1959. It was proposed as a technique capable of achieving optimal rate-distortion performance. Vector quantizer Q defined as follows:

$$Q: \mathbb{R}^d \rightarrow C \quad (9)$$

Let $C = \{c_i | i \in I, c_i \in \mathbb{R}^d\}$ denote a set of k vectors, referred to as the codebook. The code rate of the quantizer is defined as $r = (\log_2 k)/d$, representing the number of bits per component required to encode an input vector. A vector quantizer consists of two main components, the vector encoder and the vector decoder (Gersho and Gray, 1991) [24]. The encoder E maps vectors from $\mathbb{R}^d$ to an index set I , while the decoder D maps the indices back to the reproduction set C , i.e., the codebook. Therefore,

$$E: \mathbb{R}^d \rightarrow I, D: I \rightarrow C \quad (10)$$

As shown in Eq. (10), the encoder assigns the input vector to a specific partition, while the decoder retrieves the corresponding reproduction value or codeword based on the partition index. The complete operation of the VQ is expressed as:

$$Q(x) = D(E(x)) \quad (11)$$

Now, let C be represented as a matrix in which each codeword c_i corresponds to the i^{th} column of C , so that $C = [c_1, c_2, \dots, c_k] \in \mathbb{R}^{d \times k}$, where each c_i is a column vector of dimension d . Under this formulation, we obtain:

$$Q(x) = Cb \quad (12)$$

Here, let $b = [b_1, b_2, \dots, b_k]^T$, where $b_i = 1$ if $x \in S_i$; otherwise, $b_i = 0$. As evident from this definition, the vector b acts as a selector, indicating which codeword from the codebook is used to represent the input vector x .

Following the vector quantization step, a Codebook-guided self-attention (CGSA) mechanism is applied to capture long-range dependencies with linear computational complexity. Specifically, the node embeddings $H \in \mathbb{R}^{N \times d}$, generated by auxiliary message passing neural networks (MPNNs) are used as both the Query and the value in the attention mechanism. The Key $\hat{K}$ is derived from the matrices C and U . The attention operations are formally defined as:

$$Q = HW_q, V = HW_v, \quad (13)$$

$$G = \text{Norm}(\text{Linear}(C)), \quad (14)$$

$$\hat{K} = UG, \quad (15)$$

$$\hat{Z} = \text{softmax}(Q\hat{K}^T)V, \quad (16)$$

where W_q, W_v are trainable projection matrices, U is a learnable linear transformation, and $\text{Norm}(\cdot)$ denotes layer normalization.

Here d' represents the dimension of the intermediate node embeddings. In the CGSA mechanism, the Key $\hat{K}$ is computed through matrix multiplication between G and U , instead of using nearest neighbor search over the full codebook. The attention weights defined in Eq (16) can be further factorized as follows:

$$\begin{aligned} \hat{Z} &= \text{softmax}(Q\hat{K}^T)V \\ &= \text{softmax}(Q(UG)^T)V \\ &= \text{Diag}^{-1}(\exp(QC^T)U^T \mathbf{1}) \exp(QC^T)U^T V \end{aligned} \quad (17)$$

where $\mathbf{1} \in \mathbb{R}^N$ and $U^T \mathbf{1} = \{n_b\}_b^B \in \mathbb{Z}^+$ represents the number of node embeddings in C that are assigned to the same codeword. The computational complexity of CGSA is $O(NBd_v)$, where $B \ll N$.

Subsequently, auxiliary MPNNs are employed as encoders to generate node embeddings enriched with local positional information. In the CGSA architecture, the reconstructed codebook C , codeword assignments U and node embeddings H are fed into a linear-time self-attention mechanism. CGSA introduces an inductive bias by encoding global structural information through spike count embeddings. These four components are formulated as follows:

$$U^l, C^l \& = VQ^l(A), \quad (18)$$

$$H^l \& = \text{MPNN}^l(Z^{l-1}, A), \quad (19)$$

$$\hat{Z}^l \& = \text{CGSA}^l(U^l, C^l, H^l), \quad (20)$$

$$Z^l \& = \text{Linear}(\hat{Z}^l) + H^l, \quad (21)$$

$$Y \& = \text{CH}(Z^l), \quad (22)$$

The output feature representations from the GT encoder are fed into an EL mechanism to optimize the classification decision. These representations are used as input to three sub-classification models (model 1, model 2, model 3), each of which independently learns a different nonlinear classification boundary in the feature space. Due to differences in structure or training data, each model tends to learn from a

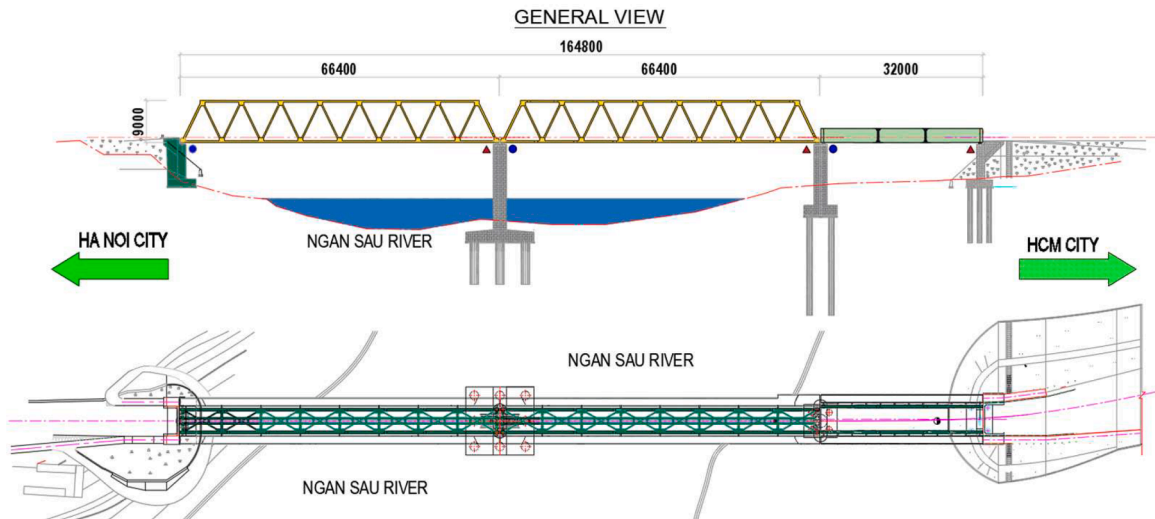


Fig. 4. Overall view of Cua Rao bridge.

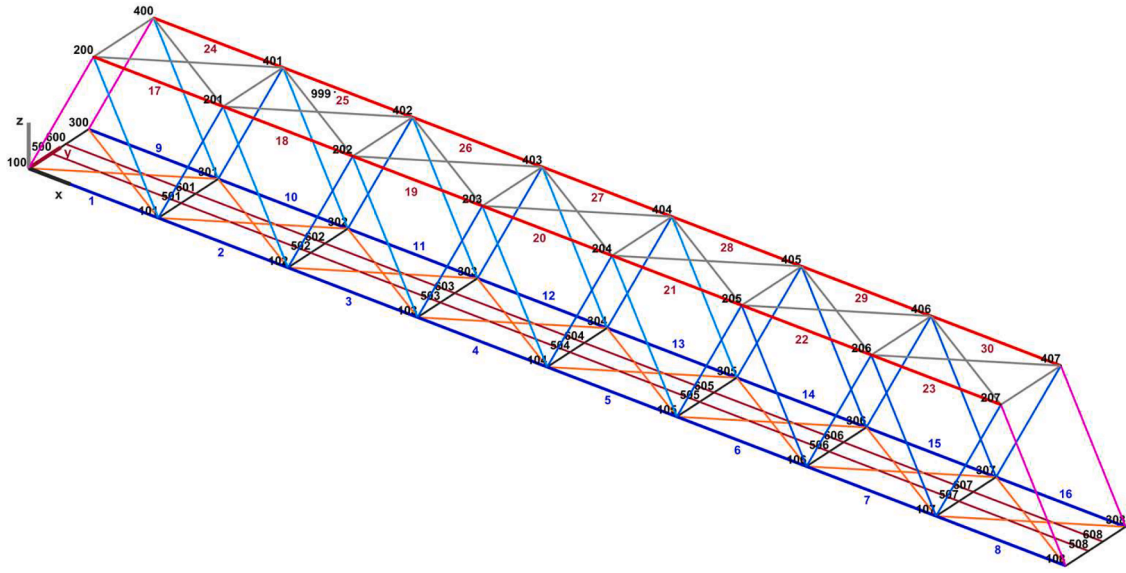


Fig. 5. FEM model of Cua Rao bridge.

different perspective, contributing to increased diversity in predictions. The outputs from the sub-models are then aggregated through an integration mechanism, which allows the prediction information from multiple models to be combined in the form of voting. In this ensemble scheme, all classifiers are assigned equal weights.

While sophisticated fusion strategies such as uncertainty-based weighting or Bayesian averaging exist, this study adopts simple majority voting to prioritize computational efficiency and real-time inference speed, which are critical requirements for practical SHM systems. Furthermore, given that structural vibration data is inherently susceptible to environmental noise, employing equal weighting serves as a regularization technique. This prevents the ensemble from overfitting to the idiosyncrasies or overconfident errors of any single sub-model, thereby ensuring greater robustness in decision-making.

The final prediction for an unlabeled instance is determined by selecting the class that receives the majority of votes. This process can be expressed mathematically as:

$$\text{Class}(x) = \underset{c_i \in \text{dom}(y)}{\text{argmax}} \sum_{\substack{y \in \text{dom}(y) \\ c_j \in \text{dom}(y)}} \text{argmax}_{c_j} P_{M_k}(y=c_j|x) \quad (23)$$

where M_k denotes classifier k , and $\hat{P}_{M_k}(y=c|x)$ denotes the probability that y takes the value c given an instance x .

3. Case study

To evaluate the effectiveness of the proposed GT-VQ-EL framework, a case study is conducted using a real-world railway bridge located in Vietnam. This case study is designed to validate the model's capability in handling realistic structural data, simulating various damage scenarios, and demonstrating the superiority of the proposed method compared to baseline approaches. The selected structure offers a representative setting for testing the performance of advanced SHM techniques under operational conditions.

3.1. Description of the Cua Rao Bridge and finite element model

The Cua Rao Bridge, located at Km 360+450 on the North–South railway line, spans the Ngan Sau River in Vu Quang District, Ha Tinh Province, Vietnam (Fig. 4). Completed in 2013, the structure comprises three simply supported steel spans: two 66.4 m steel truss spans and one 32 m steel I-beam span. The bridge has a total width of 8.8 m,

Table 1

Cases of damage and corresponding labels.

Label	Percentage of damage (%)
Class 0	0
Class 1	10
Class 2	20
Class 3	30
Class 4	40

accommodating a central single-track railway (4.8 m wide) flanked by two pedestrian walkways (2.2 m wide each).

To generate a high-fidelity dataset for training the DL models, a Finite Element Model (FEM) of the bridge was developed using the Stablib toolbox in MATLAB [25], as illustrated in Fig. 5. In this simulation, structural components were modeled as beam elements with six degrees of freedom per node, assuming rigid joint connections. The final numerical model consists of 53 nodes and 143 elements, with material properties defined to match the actual steel specifications of the bridge.

3.2. Dataset

To evaluate the diagnostic capability of the proposed model, we defined five distinct structural conditions, ranging from a healthy state to various degrees of damage (Table 1). Structural damage was simulated by reducing the stiffness (elastic modulus) of specific floor beam elements. This method effectively models common deterioration mechanisms such as cracking or corrosion, which inherently degrade load-bearing capacity and alter the bridge's dynamic characteristics.

Dataset were collected from nodes 101–107 and 301–307, which are positioned along the floor beam elements. For each node, responses were recorded in both the lateral (Y-axis) and vertical (Z-axis) directions. The data acquisition process employed a sampling rate of 1000 Hz, with a measurement duration of 50 s per node. Consequently, the raw data acquisition resulted in a continuous time-series with a shape of (5, 28, 50,000), corresponding to 5 damage scenarios, 28 input features, and 50,000 time steps. To generate a sufficient number of samples for DL training and capture localized temporal features, a sliding window technique was applied to these raw signals. A sliding window segmentation was then applied, using a window size of 700 time steps and a stride of 90, producing windowed samples of shape (28, 700) that preserve local temporal-spatial patterns while maintaining continuity of

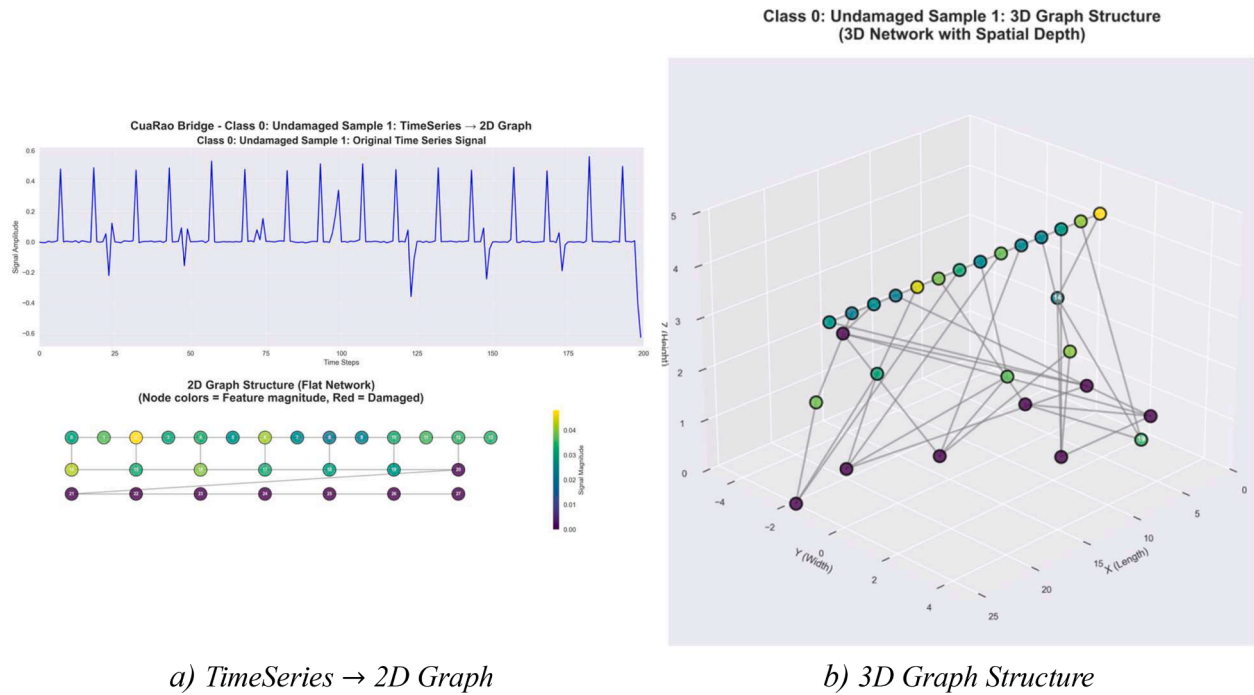


Fig. 6. Visualization of an undamaged sample (Class 0): (a) Original time-series signal and 2D topological graph; (b) 3D spatial graph structure.

the raw signal. This data augmentation process segmented the long time-series into smaller, overlapping windows, resulting in a final dataset of 2745 samples. These samples were then randomly partitioned into three subsets: 60 % for training (1647 samples), 20 % for validation (549 samples), and 20 % for testing (549 samples).

To generate the dataset, dynamic simulations were conducted under moving load conditions. A locomotive model traveling at a constant speed of 60 km/h was employed to excite the bridge structure. To approximate realistic operational environments, random noise was injected into the simulation parameters. The vibration responses were

recorded at nodes 101–107 and 301–307 (located along the floor beams) in both lateral (Y-axis) and vertical (Z-axis) directions. With a sampling rate of 1000 Hz and a duration of 50 s per simulation run, the raw data acquisition yielded a continuous time-series tensor.

4. Result and discussion

The training set was used to build and fine-tune the DL model, while the validation set was used to evaluate the model's generalization performance and detect potential overfitting during training. The test set,

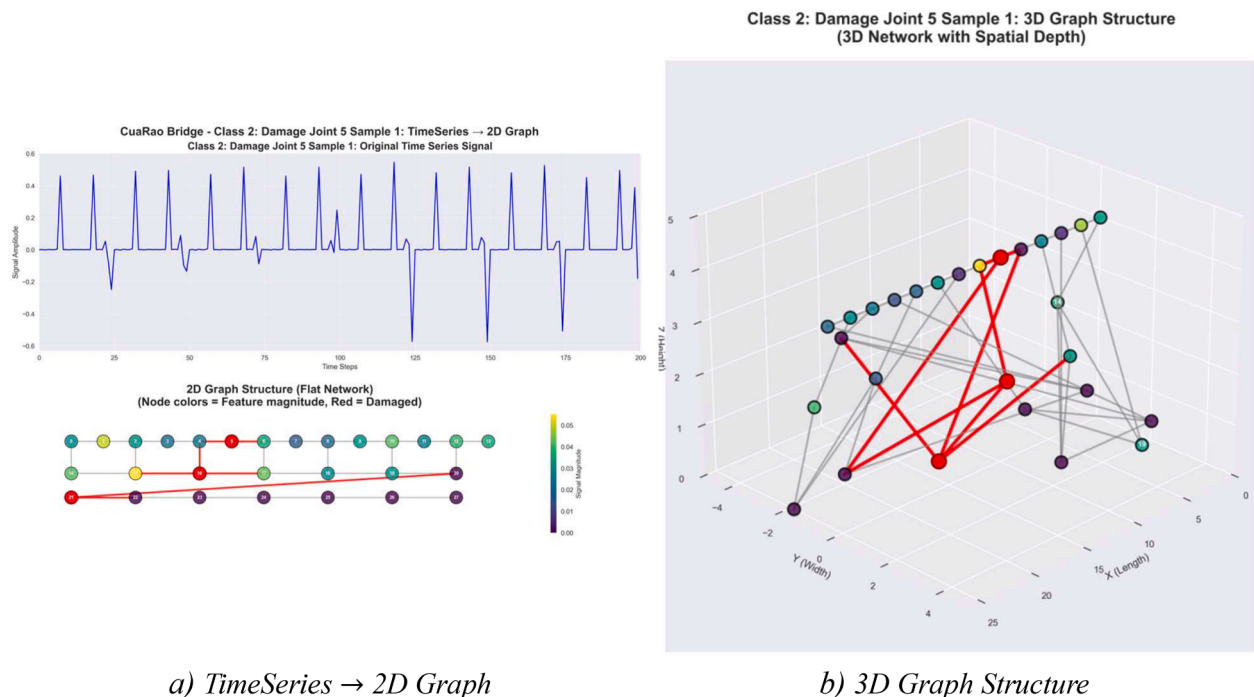


Fig. 7. Visualization of Damage Case 2 (Joint 5): (a) Time-series signal and 2D graph showing anomaly propagation; (b) 3D graph localizing the damage.

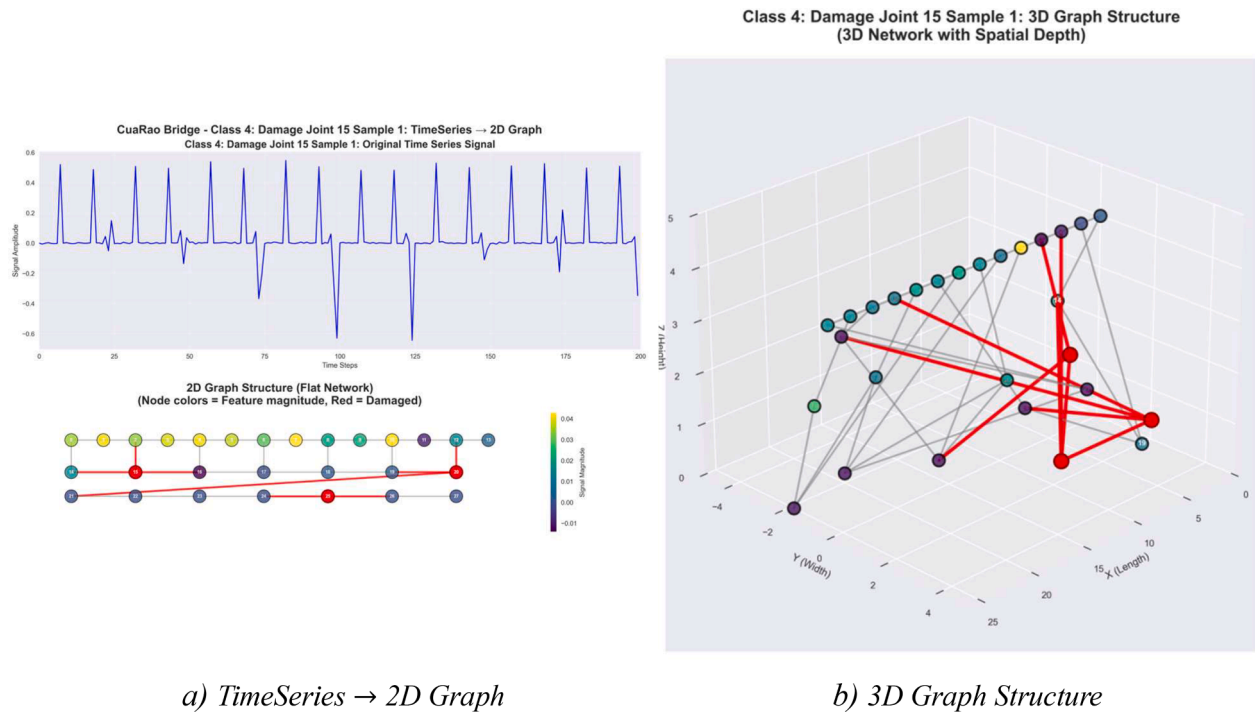


Fig. 8. Class 4: Visualization of Damage Case 4 (Joint 15): (a) Time-series signal and 2D graph; (b) 3D graph highlighting the damage location.

containing unseen data, was used exclusively for final performance assessment. This three-subset split provides a reliable and objective basis for evaluating the model's capability in realistic structural damage diagnosis tasks.

4.1. Graph-based data transformation (2D & 3D)

To capture the intrinsic structural dependencies often lost in standard one-dimensional time-series analysis, the raw vibration signals were transformed into graph-structured data. This transformation establishes a topological basis for the DL model, treating sensors as interconnected nodes rather than isolated data points. We employ two visualization modalities to interpret the feature distribution:

- **2D Graph (Topological View):** Represents the logical connectivity between sensors. Node colors encode the magnitude of extracted features, allowing for the immediate identification of high-energy anomalies.

- **3D Graph (Spatial View):** Incorporates the precise geometric coordinates (X , Y , Z) of the sensors, providing a realistic representation of signal propagation and damage localization within the bridge geometry.

Representative samples for different structural states are illustrated in Figs. 6, 7, and 8. The 3D graph representation further enriches this process by incorporating the geometric coordinates (X , Y , Z) of the sensing points. This enables the model to capture spatial hierarchies and directional patterns of signal propagation more realistically. Red-colored nodes and edges in the 3D view highlight regions with significant signal anomalies, thereby indicating localized structural damage. Examples for specific classes are shown below (Fig. 6).

As shown in Fig. 6, the undamaged state (Class 0) exhibits stable, periodic time-series responses. Consequently, the corresponding graph visualization displays a uniform feature distribution with no distinct anomalies (indicated by the absence of red nodes).

In contrast, damage scenarios introduce significant signal deviations. Fig. 7 (Class 2) depicts the structural response under 20 % stiffness

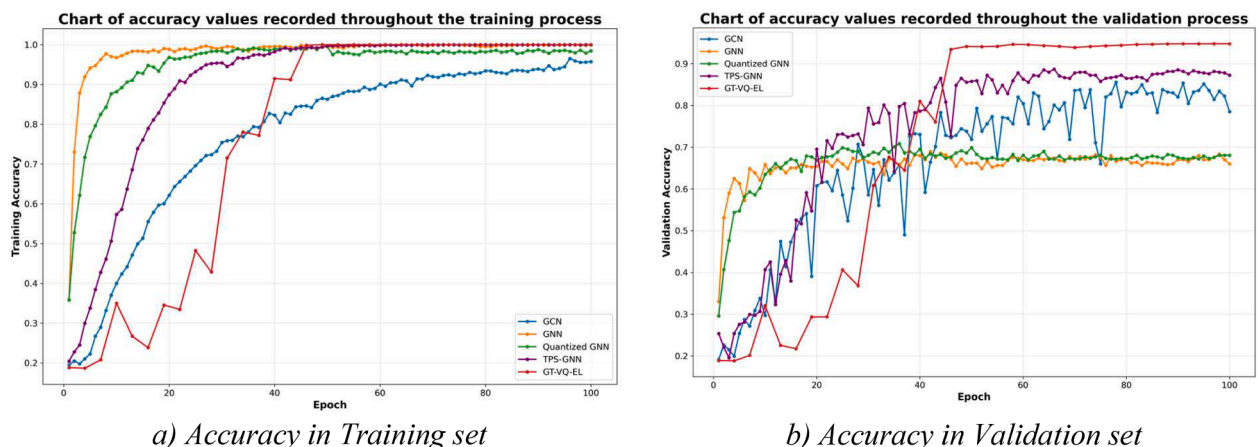


Fig. 9. Comparison of the performance accuracy of five methods.

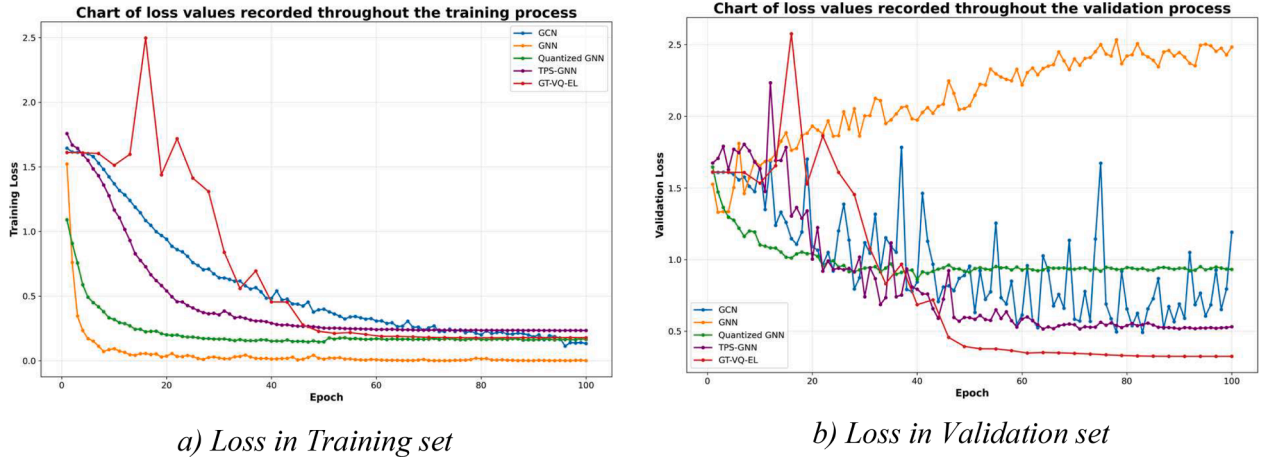


Fig. 10. Comparison of the performance loss of five methods.

reduction at Joint 5. The graph representation effectively captures this anomaly, with "red" nodes and edges clustering around the affected region (Nodes 5, 16, 21), thereby visually localizing the damage source.

Similarly, Fig. 8 (Class 4) illustrates the impact of damage at Joint 15. The high-magnitude features (red nodes 15, 20, 25) clearly shift to the new damage location. These visualizations confirm that the proposed graph-based transformation preserves crucial spatial information, enabling the model to distinguish between different damage locations based on topological feature patterns.

4.2. Convergence chart

The training dynamics of the five models were evaluated over 100 epochs using both accuracy and loss metrics. The dataset was split into training, validation, and testing subsets to rigorously assess generalization capability and detect potential overfitting.

Fig. 9 illustrates the accuracy progression. In the training phase (Fig. 9a), the proposed GT-VQ-EL model demonstrates rapid learning, reaching near-perfect accuracy (~100 %) and stabilizing quickly. The GNN and Quantized GNN models also achieve high training accuracies (98–99 %). However, their performance on the validation set (Fig. 9b) reveals a significant disparity. While GT-VQ-EL maintains a high validation accuracy of approximately 94–95 %, the baseline GNN and Quantized GNN drop significantly to 67–69 %. This large generalization gap indicates that the baseline models suffer from severe overfitting, memorizing the training data rather than learning robust features. The TPS-GNN model shows the second-best performance, exhibiting a

smooth convergence curve and stabilizing at a validation accuracy of 88–89 %. The GCN model displays higher variance (fluctuations) during training and settles at a lower validation accuracy of around 82–84 %.

Regarding validation loss, the GT-VQ-EL model (Fig. 10b) starts with a higher value but later steadily decreases with less fluctuation than the other models, indicating effective deep-learning capacity and reduced overfitting. The loss curves in Fig. 10 corroborate these findings. The GT-VQ-EL model (Fig. 10a) initially shows fluctuation but rapidly converges to a near-zero loss, reflecting an efficient optimization process. In terms of validation loss (Fig. 10b), it achieves the lowest and most stable value among all comparators. Conversely, the Quantized GNN exhibits a troubling trend where validation loss remains high and even tends to increase, further confirming its poor generalization on unseen data.

4.3. Parameters and confusion matrix for evaluating models

To quantitatively evaluate the classification performance, standard metrics including Precision, Recall, F1-score, and Accuracy were employed [26]. These are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (24)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (25)$$

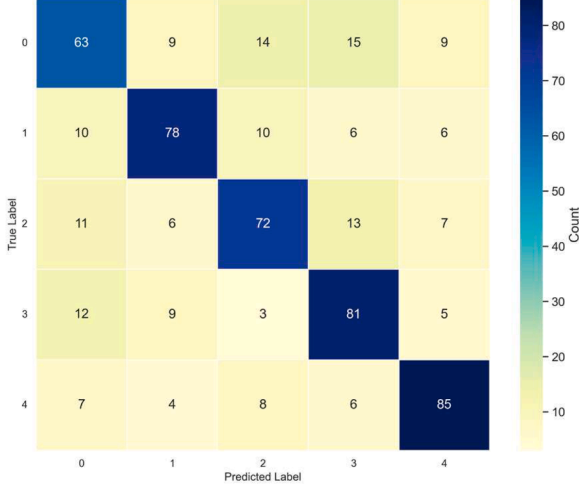
$$f1 - score = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}} \quad (26)$$

Table 2

Classification accuracy of the classes on the testing set of the five models.

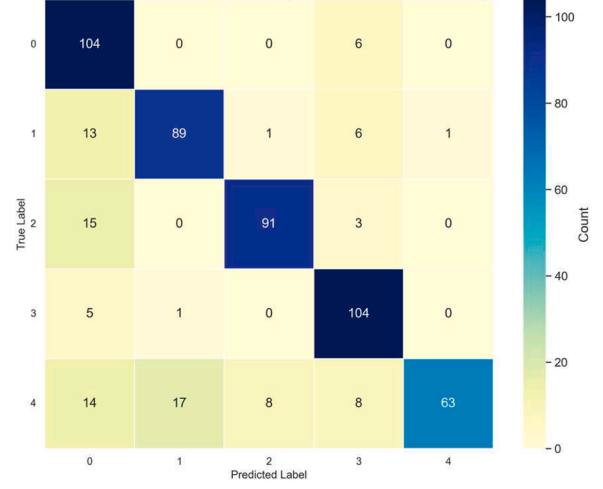
Model	GNN			GCN			Quantized GNN		
Label	Precision	Recall	f1-score	Precision	Recall	f1-score	Precision	Recall	f1-score
Class 0	0.6117	0.5727	0.5915	0.6887	0.9455	0.7969	0.6598	0.5818	0.6184
Class 1	0.7358	0.7091	0.7222	0.8318	0.8091	0.8203	0.7257	0.7455	0.7257
Class 2	0.6729	0.6606	0.6667	0.9100	0.8349	0.8708	0.6759	0.6697	0.6728
Class 3	0.6694	0.7364	0.7013	0.8189	0.9455	0.8776	0.6800	0.7727	0.7234
Class 4	0.7589	0.7727	0.7658	0.9844	0.5727	0.7241	0.8019	0.7727	0.7870
Accuracy	0.6903			0.8215			0.7086		
Model	TPS-GNN			GT-VQ-EL					
Label	Precision	Recall	f1-score	Precision	Recall	f1-score			
Class 0	0.9286	0.8273	0.8750	0.9904	0.9364	0.9626			
Class 1	0.9216	0.8545	0.8868	0.9434	0.9091	0.9259			
Class 2	0.7926	0.9817	0.8770	0.9524	0.9174	0.9346			
Class 3	0.9145	0.9727	0.9427	0.8983	0.9636	0.9298			
Class 4	0.9588	0.8455	0.8986	0.9052	0.9545	0.9292			
Accuracy	0.8962			0.9362					

Confusion matrix of the GNN method applied to the CuaRao bridge dataset.



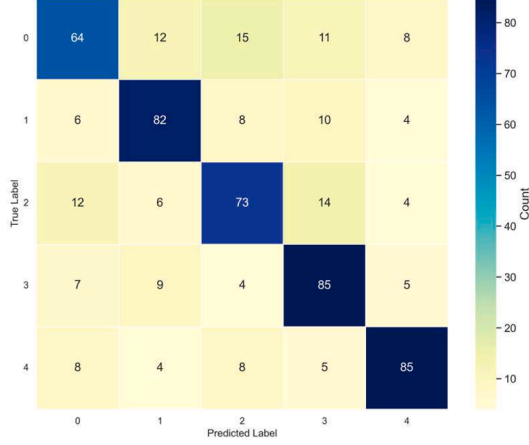
a) GNN

Confusion matrix of the GCN method applied to the CuaRao bridge dataset.



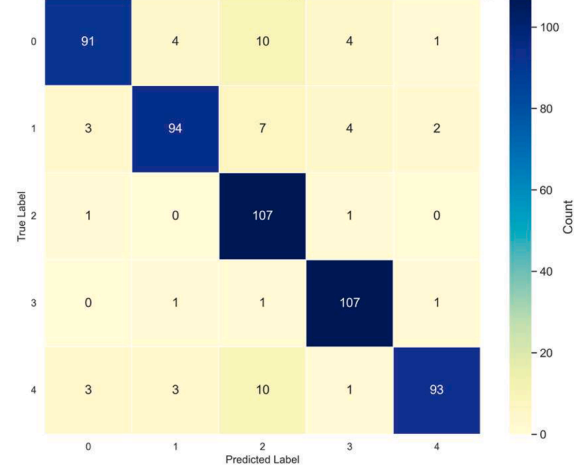
b) GCN

Confusion matrix of the Quantized GNN method applied to the CuaRao bridge dataset.



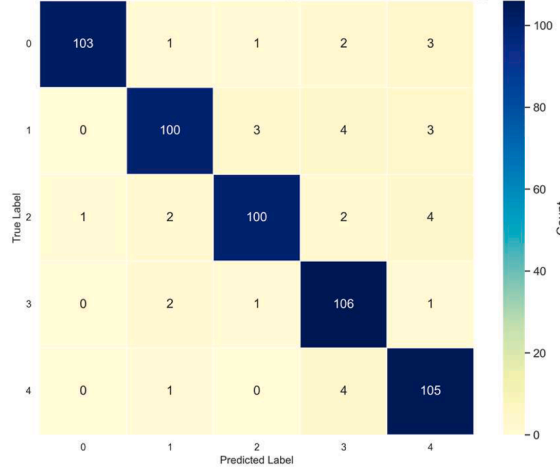
c) Quantized GNN

Confusion matrix of the TPS-GNN method applied to the CuaRao bridge dataset.



d) TPS-GNN

Confusion matrix of the GT-VQ-EL method applied to the CuaRao bridge dataset.



e) GT-VQ-EL

Fig. 11. Confusion Matrix of five models.

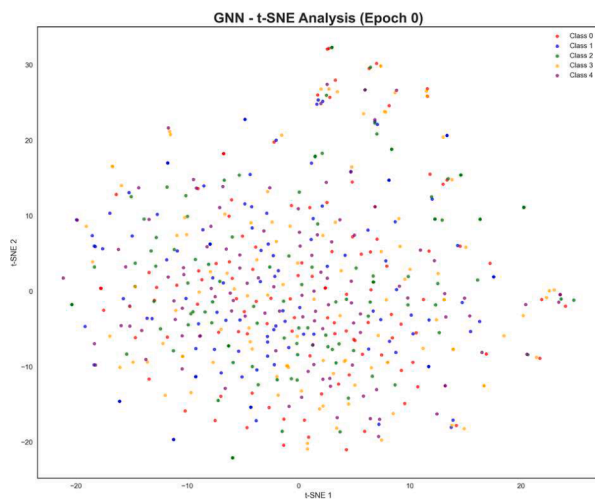
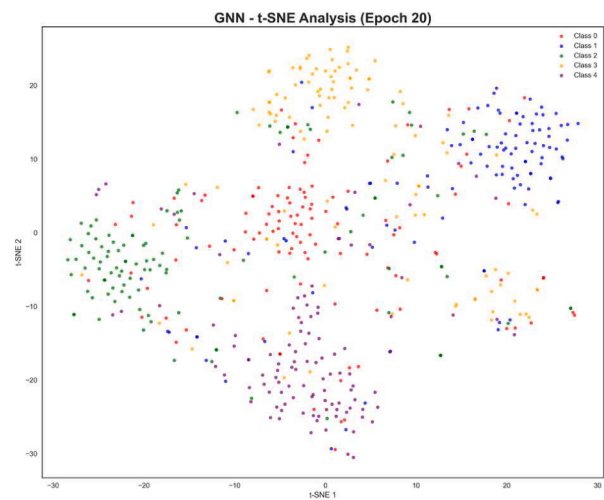
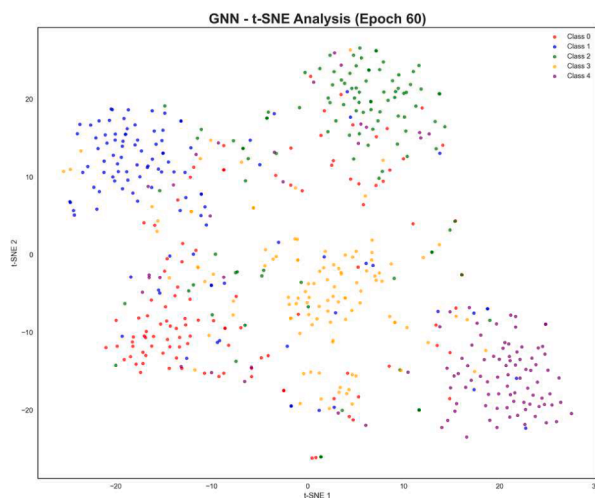
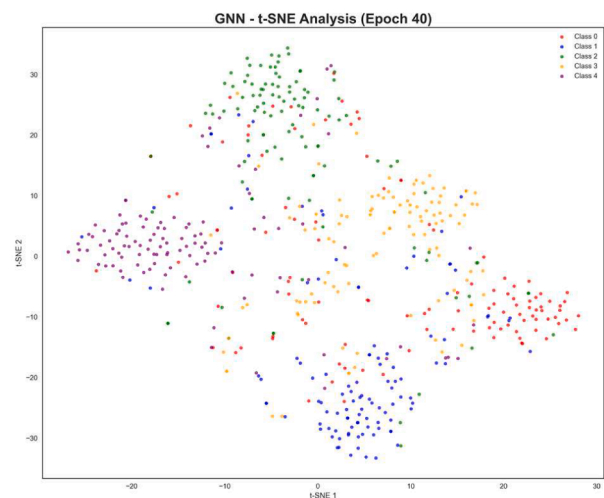
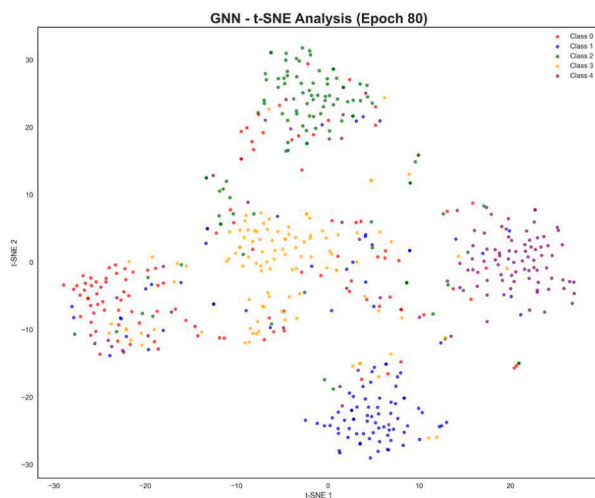
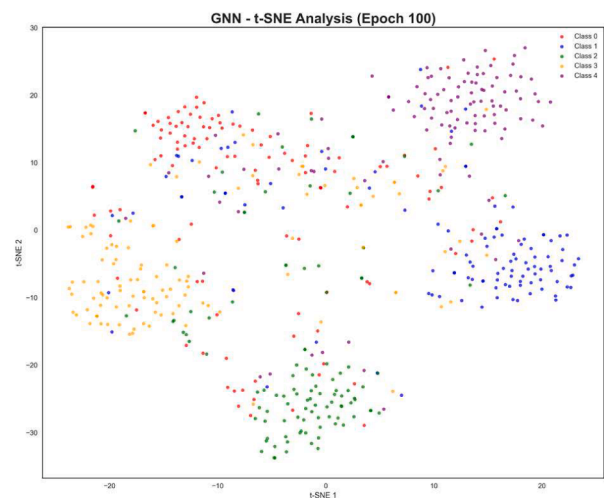
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (27)$$

where TP, TN, FP, and FN denote True Positives, True Negatives, False

Positives, and False Negatives, respectively.

Table 2 shows the evaluation metrics based on ground truth, including precision, recall, and F1-score for each model.

The proposed GT-VQ-EL model demonstrates superior performance

*a) Epoch 0**b) Epoch 20**c) Epoch 40**d) Epoch 60**e) Epoch 80**f) Epoch 100***Fig. 12.** Model GNN-t-SNE analysis.

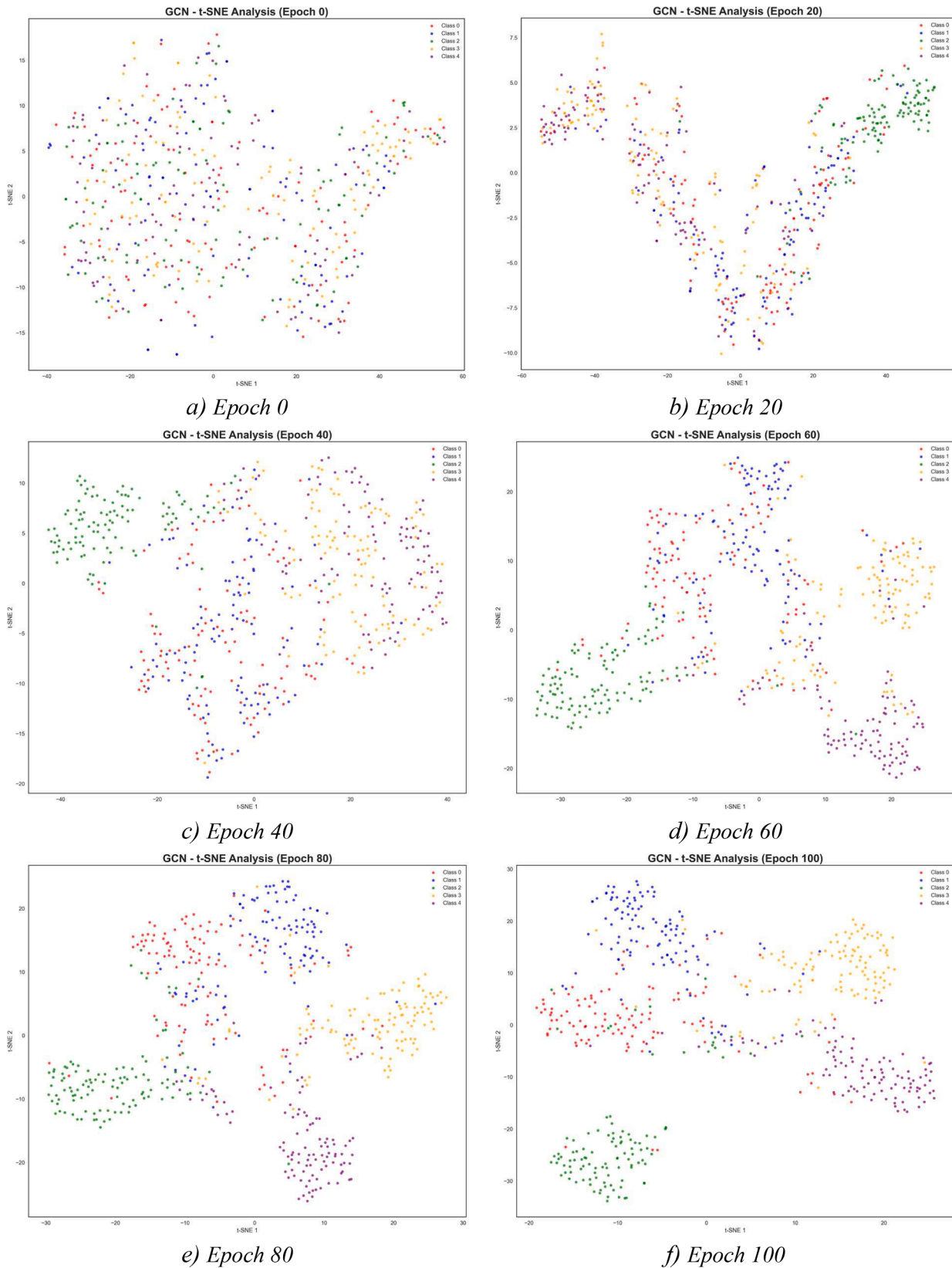


Fig. 13. Model GCN-t-SNE analysis.

across all metrics, achieving the highest overall accuracy of 93.62 %. Notably, it maintains high F1-scores across all damage classes (ranging from 0.9259 to 0.9626), indicating a balanced predictive capability without bias toward any specific state. The TPS-GNN model follows with

89.62 % accuracy but exhibits slight instability in precision for Class 2. In contrast, the baseline models (GNN, GCN, Quantized GNN) show significantly lower performance, particularly in distinguishing the healthy state (Class 0), where F1-scores drop as low as 0.59.

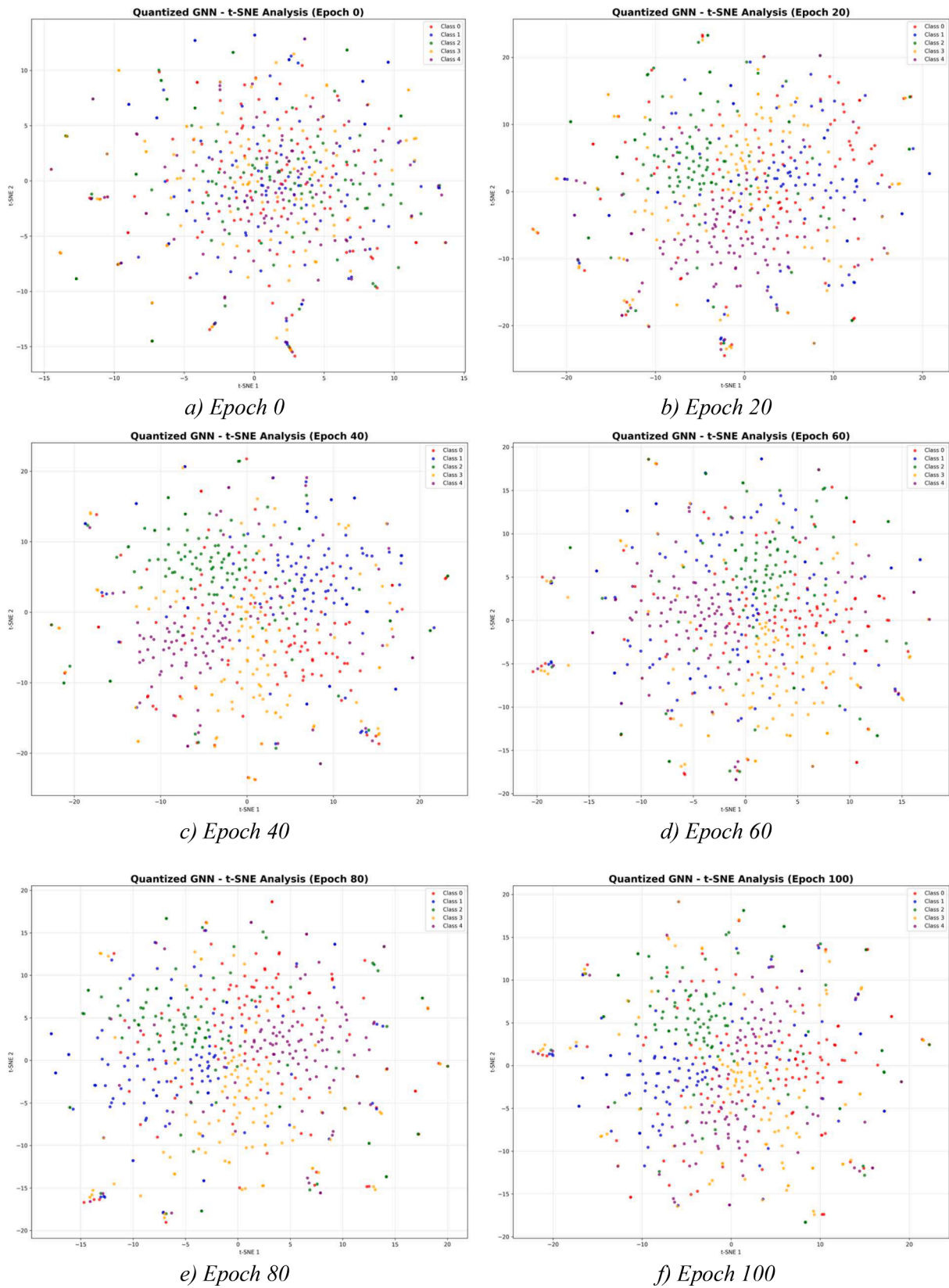


Fig. 14. Model Quantized GNN-t-SNE Analysis.

The confusion matrices presented in Fig. 11 provide deeper insights into the specific misclassification patterns of each model. The proposed GT-VQ-EL model (Fig. 11e) exhibits a strong diagonal dominance characterized by minimal off-diagonal noise. Its misclassifications are

notably rare and predominantly confined to adjacent damage levels, such as between Class 1 and Class 2, which is physically understandable given the similarity in structural response at these stages. In comparison, while the TPS-GNN model (Fig. 11d) demonstrates solid overall

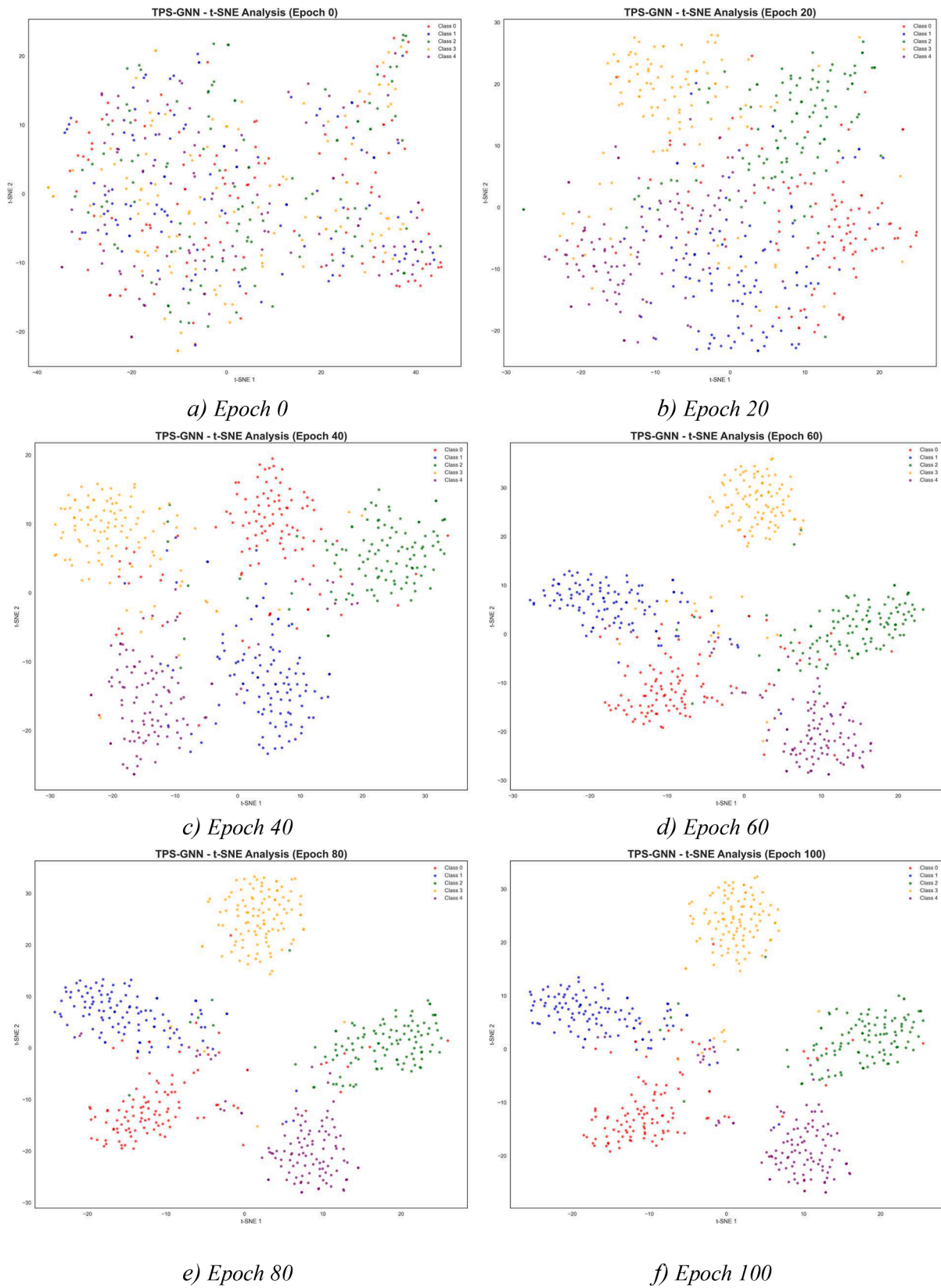


Fig. 15. Model TPS-GNN-t-SNE Analysis.

performance, it still exhibits discernible confusion between healthy samples (Class 0) and those with minor damage (Class 1 or 2). Conversely, the baseline models, specifically GNN and Quantized GNN (Fig. 11a–c), reveal significant classification errors, most notably

between Class 0 and Class 2, as well as Class 1 and Class 3.

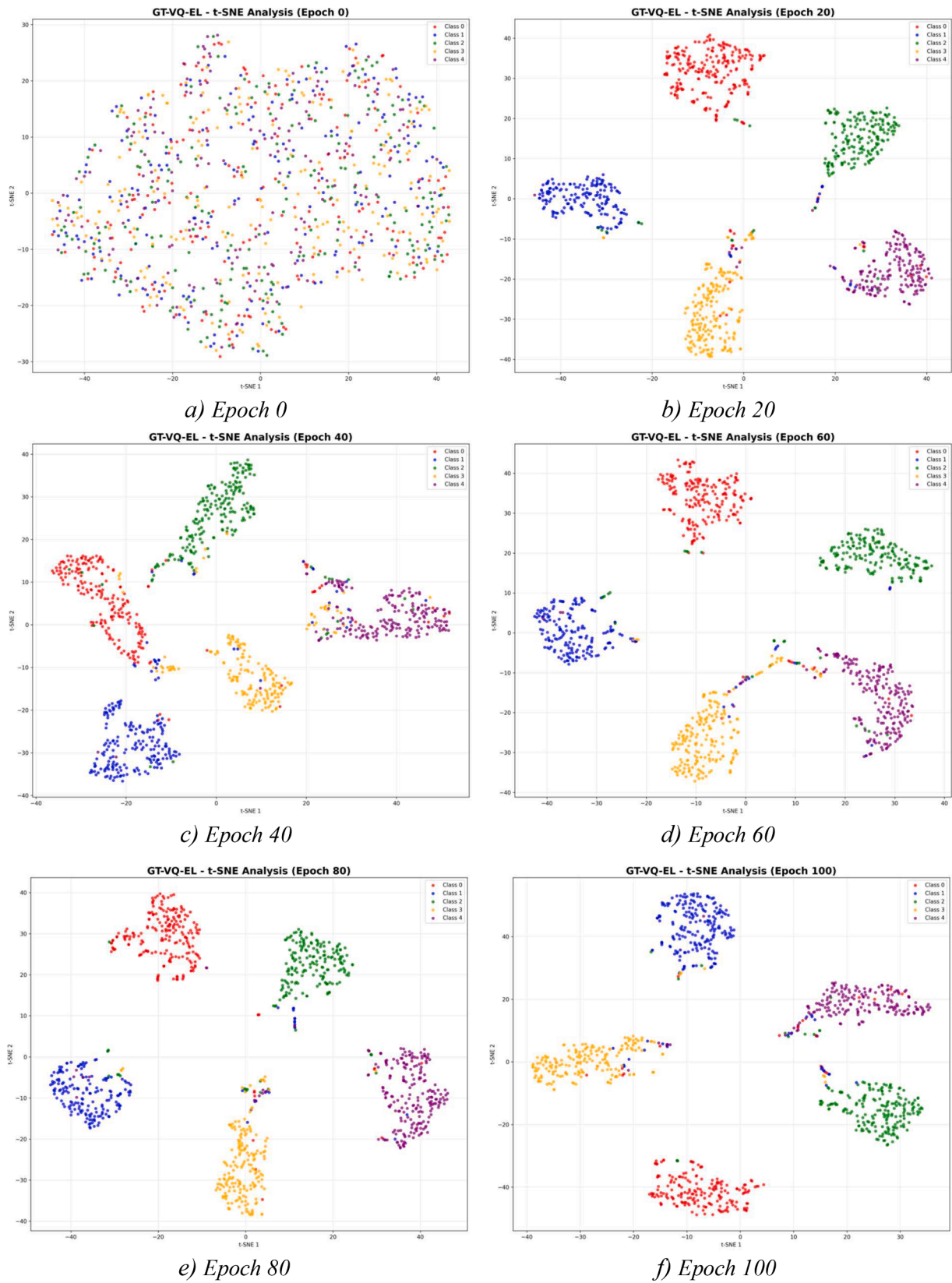


Fig. 16. Model GT-VQ-EL-t-SNE Analysis.

4.4. The t-SNE visualizations

4.4.1. GNN model. The t-SNE visualizations for the GNN model (Fig. 12) show gradual improvement in class separation throughout

training. At epoch 0, the classes are heavily overlapped. By epoch 20, loose clusters appear, but significant confusion remains, especially between Class 0 and Class 1. At epoch 40, the clusters become more distinguishable but with overlaps among Classes 0, 2, and 3. From epoch

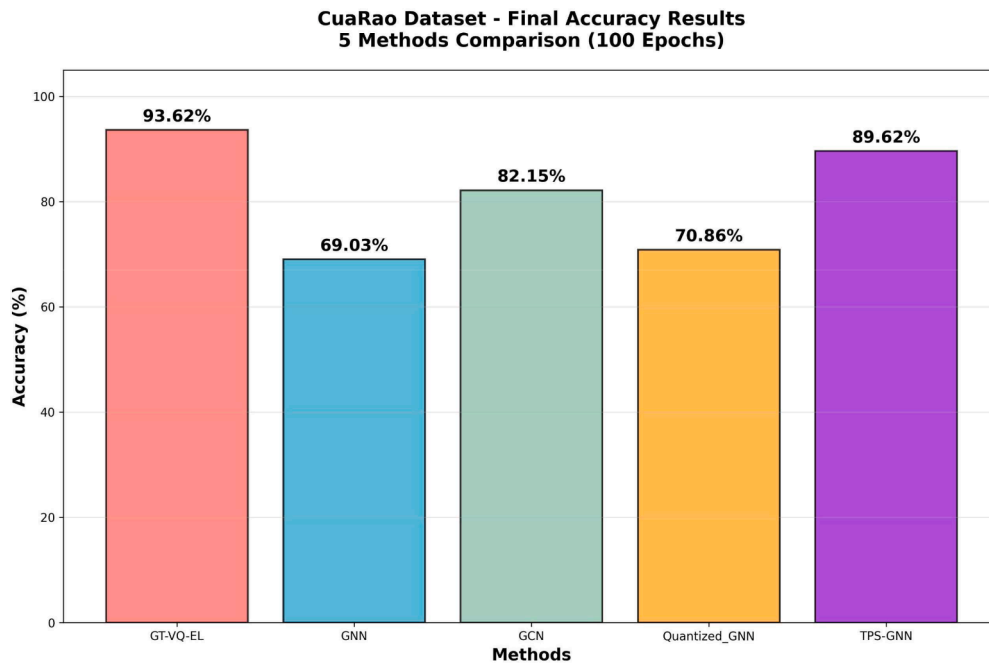


Fig. 17. CuaRao - five methods complete comparison.

60 onward, separation is clearer, with Class 3 and Class 4 forming well-defined groups. By epoch 100, all five classes are stably clustered with minimal overlap, demonstrating convergence and effective feature discrimination.

4.4.2. GCN model. The t-SNE visualizations for the GCN model (Fig. 13) highlight its faster convergence and stronger class separability compared to GNN. By epoch 20, the samples already begin to align into emerging clusters, with class 4 showing clearer grouping than others. At epoch 40, all five classes start forming well-organized clusters, with class 3 and class 4 particularly well separated. From epoch 60 onward, the embeddings become increasingly compact, and by epoch 100, the clusters become highly stable with sharp boundaries, indicating that GCN learns class-specific features more effectively and in fewer epochs than GNN.

4.4.3. Quantized GNN model. The t-SNE visualizations for the Quantized GNN model (Fig. 14) reveal limited improvement in class separability throughout training. At epoch 0, the samples are randomly scattered without any visible clustering. Even from Epoch 20 to Epoch 40, the embeddings remain highly overlapped, with only weak grouping trends. From epoch 60 onward, the clusters show minor improvements, but all five classes continue to exhibit substantial overlap, especially among Classes 0, 2, and 3. Unlike GNN and GCN, the Quantized GNN model fails to establish clear and stable boundaries, indicating unstable feature learning and poor discriminative representation.

4.4.4. TPS-GNN model. The t-SNE visualizations of the TPS-GNN model (Fig. 15) illustrate a clear and progressive improvement in class separability as training advances. At Epoch 0, the feature embeddings are randomly dispersed, showing no recognizable structure or class-wise organization. By Epoch 20, preliminary grouping patterns start to emerge, although the boundaries between classes remain weak and several clusters still partially overlap. A significant improvement becomes evident from Epoch 40 onward, where distinct cluster formation begins to appear, with each class gradually occupying a more compact and coherent region in the embedding space. At Epoch 60, the separation becomes markedly clearer. This trend continues through Epochs 80 and 100, where the class clusters stabilize into well-defined and spatially

Table 3

Statistical comparison between GT-VQ-EL and GCN.

No.	Metric	Value
1	Number of test samples	549
2	GT-VQ-EL accuracy	93.62 %
3	GCN accuracy	82.15 %
4	Accuracy difference (GT-VQ-EL – GCN)	0.1148
5	McNemar chi-square statistic	30.2677
6	McNemar p-value	< 0.00001
7	Bootstrap mean accuracy difference	0.1147
8	Bootstrap standard deviation	0.0198
9	Bootstrap 95 % CI for difference	[0.0765, 0.1530]
10	Bootstrap p-value	< 0.00001

separated groups.

4.4.5. GT-VQ-EL model. The t-SNE visualizations for the GT-VQ-EL model (Fig. 16) clearly demonstrate its superior capability in feature learning and class separation. At epoch 0, the data points are distributed in an overlapped manner without forming distinct clusters. However, by epoch 20, the model rapidly organizes the embeddings into five well-defined and clearly separated clusters. From epoch 40 onward, the clusters become increasingly compact and stable, with minimal overlap across classes. By epoch 100, the embeddings achieve complete and robust separation, reflecting both convergence and strong generalization ability. Compared to GNN, GCN, and Quantized GNN, GT-VQ-EL outperforms the other models in both feature organization and class distinction, confirming it as the most powerful and effective model in the evaluation, particularly in the context of time-series classification.

4.5. Statistical significance and robustness validation

While the accuracy comparisons in previous sections highlight the performance gap between models, it is crucial to verify that these improvements are statistically significant and not merely results of random variation. Fig. 17 provides a final summary of the test accuracies after 100 epochs, where the proposed GT-VQ-EL model (93.62 %) demonstrates a clear lead over the baselines, surpassing the standard GCN (82.15 %) by a margin of 11.47 %

Table 4
McNemar contingency table between GCN and GT-VQ-EL.

Model	GT-VQ-EL correct	GT-VQ-EL wrong
GCN correct	419	32
GCN wrong	95	3

To rigorously validate this performance superiority, we conducted two statistical tests on the test set ($N=549$): McNemar’s test and Bootstrap analysis. The comparison focuses on GT-VQ-EL versus GCN, establishing the proposed framework’s improvement over standard graph convolutional approaches.

As detailed in Table 3, McNemar’s test yielded a Chi-square statistic of $\chi^2=30.2677$ with a p-value <0.00001 , which is well below the standard significance level of $\alpha=0.05$. This result strongly rejects the null hypothesis, confirming that the predictive difference between the two models is statistically significant.

The contingency table (Table 4) further reveals the asymmetry in error patterns: GT-VQ-EL correctly classified 95 samples that GCN misclassified, whereas GCN outperformed GT-VQ-EL in only 32 cases.

Finally, to assess the stability of this improvement, a Bootstrap analysis with 10,000 iterations was performed. The results show a mean accuracy difference of 11.47 % with a 95 % Confidence Interval (CI) of [7.65 %, 15.30 %]. Crucially, since this interval does not contain zero, it statistically confirms that the GT-VQ-EL model consistently and robustly outperforms the GCN baseline across different data resamplings.

5. Conclusion

This study proposed a novel DL framework, GT-VQ-EL, designed for robust structural damage diagnosis using time-series vibration data. By synergizing the spatiotemporal modeling capabilities of GT, the noise-filtering properties of VQ, and the decision stability of EL, the proposed architecture effectively overcomes the limitations of conventional graph-based models. Specifically, it successfully captures long-range structural dependencies while mitigating the adverse effects of environmental noise and measurement uncertainty.

Extensive consideration was conducted on a dataset generated from a calibrated FEM of the Cua Rao Bridge, incorporating diverse damage scenarios. The results demonstrate that GT-VQ-EL achieves a superior classification accuracy of 93.62 % and a macro F1-score of 0.9626, significantly outperforming baseline architectures such as GNN, GCN, Quantized GNN, and TPS-GNN. Furthermore, statistical validation confirms that the proposed model offers faster convergence, enhanced robustness, and greater generalization capability compared to individual baseline models.

Despite these promising results, this study acknowledges certain limitations. The current validation relies on synthetic data derived from numerical simulations, which, although calibrated, may not fully capture the complex environmental variability (e.g., temperature shifts, wind loads) encountered in operational settings. Furthermore, although 2D and 3D transformation techniques are used to increase the diversity of the data, they might not fully capture the complexity of environmental fluctuations and actual operations. Consequently, future research will focus on two key directions: (1) validating the framework on large-scale real-world bridge monitoring datasets to assess its transferability and practical applicability; and (2) exploring model optimization and compression techniques to reduce computational complexity, thereby facilitating real-time deployment on edge computing devices for continuous SHM.

CRediT authorship contribution statement

Hung Tran Viet: Writing – review & editing, Validation, Methodology. **Hoa Tran Ngoc:** Writing – review & editing, Validation, Supervision, Methodology. **Bui Tien Thanh:** Resources, Funding acquisition,

Data curation. **Tuyen Vu Quang:** Software, Methodology, Data curation. **Loc Bui Phuc:** Software, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Hoa Tran Ngoc was funded by the Postdoctoral Scholarship Programme of Vingroup Innovation Foundation (VINIF), code VINIF.2024.STS.07.

Data availability

Data will be made available on request.

References

[1] Y.-J. Cha, R. Ali, J. Lewis, O. Büyükcöztürk, Deep learning-based structural health monitoring, *Autom. Constr.* 161 (2024) 105328, <https://doi.org/10.1016/j.autcon.2024.105328>.

[2] L. Neumann, P. Wittenberg, A. Mendler, J. Gertheiss, Confounder-adjusted covariances of system outputs and applications to structural health monitoring, *Mech. Syst. Signal. Process.* 224 (2025) 111983, <https://doi.org/10.1016/j.ymssp.2024.111983>.

[3] M.M. Azad, Y. Cheon, I. Raouf, S. Khalid, H.S. Kim, Intelligent Computational Methods for Damage Detection of Laminated Composite Structures for Mobility Applications: a Comprehensive Review, *Arch. Computat. Methods Eng.* 32 (1) (Jan. 2025) 441–469, <https://doi.org/10.1007/s11831-024-10146-y>.

[4] T.-V. Manh, H.-T. Ngoc, M.-T. Duc, L.-B. Phuc, L.-N. Duc, An Effective Damage Detection Approach for a Truss Bridge Using a Hybrid Deep Learning Model, in: *Proceedings of the 5th International Conference on Sustainability in Civil Engineering - Volume 2*, vol. 663, in Lecture Notes in Civil Engineering 663, Springer Nature Singapore, Singapore, 2025, pp. 91–101, https://doi.org/10.1007/978-981-96-5206-8_10.

[5] L.H. Viet, T.T. Thi, B.H. Xuan, Swarm intelligence-based technique to enhance performance of ANN in structural damage detection, *Transp. Commun. Sci. J.* 2022 73 (1) (2022) 1–15, <https://doi.org/10.47869/tcsj.73.1.1>.

[6] L.N. Ngoc, Q.N. Huu, L.N. Ngoc, H.N. Tran, Performance evaluation of the artificial hummingbird algorithm in the problem of structural damage identification, *Transp. Commun. Sci. J.* 74 (4) (2023) 413–427, <https://doi.org/10.47869/tcsj.74.4.3>.

[7] L. Nguyen-Ngoc, et al., A two-step approach for damage identification in bridge structure using convolutional Long Short-Term Memory with augmented time-series data, *Adv. Eng. Softw.* 198 (2024) 103795, <https://doi.org/10.1016/j.advengsoft.2024.103795>.

[8] T.B. Tien, T.V. Quang, L.N. Ngoc, H.T. Ngoc, Enhancing time series data classification for structural damage detection through out-of-distribution representation learning, *Structures*, Elsevier, 2024 106766, <https://doi.org/10.1016/j.istruc.2024.106766>.

[9] H.V. Tran, T.C. Nguyen, T.X. Le, N.T.-C. Nguyen, T.V. Ngo, H.N. Tran, Enhancing Salp Swarm Optimization with Orthogonal Diagonalization Transformation for Damage Detection in Truss Bridge, *KSCSE J. Civ. Eng.* 28 (6) (Jun. 2024) 2355–2365, <https://doi.org/10.1007/s12205-024-2048-6>.

[10] F. Kosova, Ö. Altay, H.Ö. Ünver, Structural health monitoring in aviation: a comprehensive review and future directions for machine learning, *Nondestruct. Test. Eval.* 40 (1) (Jan. 2025) 1–60, <https://doi.org/10.1080/10589759.2024.2350575>.

[11] R. Katam, V.D.K. Pasupuleti, P. Kalapatapu, Machine learning-driven structural health monitoring: sTFT-based feature extraction for damage detection, *Structures*, Elsevier, 2025 109244, <https://doi.org/10.1016/j.istruc.2025.109244>.

[12] H. Jiang, et al., Nonlinear time-dependent mapping model for shm data utilizing gan-based imputation and LSTM for uncertainty quantification, *Structures*, Elsevier, 2025 108290, <https://doi.org/10.1016/j.istruc.2025.108290>.

[13] G. Dadoulis, G.D. Manolis, K. Katakalos, K. Dragos, K. Smarsly, Damage detection in lightweight bridges with traveling masses using machine learning, *Eng. Struct.* 322 (2025) 119216, <https://doi.org/10.1016/j.engstruct.2024.119216>.

[14] B.N. Dung, M.D. Lai, T.V. Hieu, N.B. TH, Multiple vehicles detection and tracking for intelligent transport systems using machine learning approaches, *Transp. Commun. Sci. J.* 70 (3) (2019) 214–224, <https://doi.org/10.25073/tcsj.70.3.29>.

[15] Y. Liu, L. Zhang, J. Xin, S. Peng, TPS-GNN: predictive model for bridge health monitoring data based on temporal periodicity and global spatial distribution characteristics, *Struct. Health Monit.* (Apr. 2025) 14759217251330971, <https://doi.org/10.1177/14759217251330971>.

[16] X. Li, L. Xu, H. Guo, L. Yang, Application of graph convolutional neural networks combined with single-model decision-making fusion neural networks in structural

- damage recognition, *Sensors* 23 (23) (2023) 9327, <https://doi.org/10.3390/s23239327>.
- [17] H.T. Kose, J. Nunez-Yanez, R. Piechocki, J. Pope, A survey of Computationally Efficient graph neural networks for reconfigurable systems, *Information* 15 (7) (2024) 377, <https://doi.org/10.3390/info15070377>.
- [18] L.D. Lingle, "Transformer-VQ: linear-Time Transformers via Vector Quantization," Feb. 25, 2024, *arXiv: arXiv:2309.16354*. doi:10.48550/arXiv.2309.16354.
- [19] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, 2008, doi:10.1109/TNN.2008.2005605.
- [20] B. Khemani, S. Patil, K. Kotecha, S. Tanwar, A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions, *J. Big. Data* 11 (1) (Jan. 2024) 18, <https://doi.org/10.1186/s40537-023-00876-4>.
- [21] T.N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," Feb. 22, 2017, *arXiv: arXiv:1609.02907*. doi:10.48550/arXiv.1609.02907.
- [22] L.B. Saad, B. Beferull-Lozano, Quantization in graph convolutional neural networks, in: 2021 29th European Signal Processing Conference (EUSIPCO), IEEE, 2021, pp. 1855–1859, <https://doi.org/10.23919/EUSIPCO54536.2021.9615973>.
- [23] C.E. Shannon, Coding theorems for a discrete source with a fidelity criterion institute of radio engineers, international convention record, vol. 7, 1959, Claude E. Shannon: Collected Papers (1993) 325–350, <https://doi.org/10.1109/9780470544242.ch21>.
- [24] A. Gersho, R.M. Gray, *Vector Quantization and Signal Compression*, 159, Springer Science & Business Media, 2012.
- [25] D. Doms et al., "StaBIL: a finite element toolbox for MATLAB," *VERSION 2.0 USER'S GUIDE*, 2010.
- [26] D. Chicco, G. Jurman, The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, *BMC. Genomics*. 21 (1) (Dec. 2020), <https://doi.org/10.1186/s12864-019-6413-7>.